

TD/TP 9 CODAGE HUFFMANN

Exercice 1

On souhaite compresser les textes composés uniquement de 5 caractères d'alphabet $\mathcal{A} = \{a, b, c, d, e\}$. Et soit un texte composé de 100 caractères dont les fréquences d'apparition des caractères de $\mathcal{A}$ sont : $\text{frec}(a) = 20$, $\text{frec}(b) = 10$, $\text{frec}(c) = 5$, $\text{frec}(d) = 40$, $\text{frec}(e) = 25$.

1) Décrire la table de codage des caractères de $\mathcal{A}$ avec les deux méthodes suivantes :

a) Codage de longueur fixe.

b) Codage de longueur variable :

- Décrire graphiquement les différentes étapes de la construction de l'arbre de Huffman.
- Décrire les structures de données pour **implémenter** la création de l'arbre de Huffman.
- En déduire le contenu de ses structures à partir de la question précédente.
- Quelle est la structure de donnée intermédiaire utilisée pour la création de l'arbre Huffman ? (Justifier votre réponse).
- En utilisant cette structure, décrire étape par étape l'évolution des structures de données représentant l'arbre de Huffman.

2) Calculer la taille du texte pour chacune des méthodes de codages ci-dessus. En déduire le taux de compression.

3) Appliquer pas à pas la fonction de codage vue en cours pour déterminer le code des caractères de $\mathcal{A}$.

4) Expliquer le principe de codage du mot suivant « ccddae ».

5) Expliquer le principe de décodage du code suivant : « 1111001110001101001010 ».

Exercice 2

Réécrire les opérations pour la gestion d'un TasMin en tenant compte de la priorité des éléments qui est la fréquence des caractères pour un message donné.

Note : Pour faciliter l'implémentation de l'arbre de Huffman, on suppose que les données sont du TasMin sont les indices du tableau de l'alphabet $\mathcal{A} = \{a, b, c, d, e\}$ du message à coder et que la priorité des éléments du TasMin est définie dans un tableau de fréquence. Le tableau de fréquence est indicé par les indices représentant les caractères de l'alphabet $\mathcal{A}$.

TP

En utilisant l'algorithme de Huffman, écrire un programme C qui permet à l'utilisateur de compresser et décompresser des messages. Pour simplifier l'implémentation, on suppose dans un premier temps que le message est **une chaîne de caractères** composée uniquement de l'alphabet $\mathcal{A} = 'a'..'e'$; puis dans un second temps le message est un **fichier texte**.

ANNEXE : MANIPULATIONS DE BASE DES FICHIERS EN C

Les fichiers sont représentés dans les programmes C par le type `FILE`, défini dans `<stdio.h>`

Un fichier se manipule en trois étapes classiques :

1. Ouverture du fichier. Cela établit le lien entre la variable du programme (de type `FILE*` en général) et le fichier physique.
2. Lectures, écritures et déplacements. En fonction du type de fichier (texte ou binaire), on aura recours à deux fonctions différentes pour les lectures/écritures.
3. Fermeture du fichier. Cela rompt le lien établi à l'ouverture, et garantit que les modifications apportées au fichier pendant l'étape 2 seront bien écrites sur le disque.

Ouverture d'un fichier

L'ouverture se fait à l'aide de la fonction `fopen`:

```
FILE* fopen(char* nomFichier, char* mode);
```

le mode précise les opérations qu'on souhaite faire sur le contenu du fichier, comme résumé dans le tableau suivant:

mode	signification
r	lecture seule
w	écriture seule. N.B. : RAZ du fichier!
a	écriture seule, après les données existant dans le fichier
r+	lecture et écriture, positionnement en début de fichier
w+	lecture et écriture, mais RAZ du fichier au départ!
a+	lecture et écriture, positionnement en fin de fichier

Lecture / écriture de fichier "texte"

On utilise les fonctions **fscanf** et **fprintf**, similaires à `scanf` et `printf` dans leur comportement:

- `int fscanf(FILE* fichier, char* format, ...);`
Cette fonction renvoie le nombre d'éléments effectivement lus, ou EOF si aucune lecture n'était possible.
- `int fprintf(FILE* fichier, char * format, ...);`
Cette fonction renvoie le nombre de caractères effectivement écrits (sans le `'\0'` final)

Lecture / écriture de fichier binaire

On utilise les fonctions **fread** et **fwrite** suivantes:

- `size_t fread( void *buffer, size_t tailleUnitaire, size_t nbElts, FILE * fichier);`
Cette fonction lit *nbElts* objets de taille *tailleUnitaire* dans le fichier *fichier*, et stocke leur valeur dans la zone pointée par *buffer*. Elle renvoie le nombre d'objets effectivement lus ($\leq nbElts$)
- `size_t fwrite( const void *buffer, size_t tailleUnitaire, size_t nbElts, FILE *fichier);`
Cette fonction écrit dans *fichier* *nbElts* objets de taille *tailleUnitaire*. Ces objets ont été placés à l'adresse *buffer* au préalable. Elle renvoie le nombre d'objets effectivement écrits ($\leq nbElts$)

Déplacements dans un fichier

On utilise les fonctions **fseek** et **ftell** pour, respectivement, se déplacer et savoir où on est dans un fichier:

- `int fseek( FILE * fichier, long offset, int base);`
Cette fonction déplace la tête de lecture / écriture du fichier de *offset* octets à partir de *base*. *base* peut prendre trois valeurs (`SEEK_SET`, début du fichier - `SEEK_CUR`, position courante - `SEEK_END`, fin du fichier). Elle renvoie 0 en cas de succès et -1 en cas d'échec.
- `long ftell( FILE * fichier);`
Cette fonction renvoie la position courante de la tête de lecture/écriture dans le fichier.

Fermeture de fichier

On utilise la fonction **fclose** :

- `int fclose( FILE * fichier);`
Cette fonction renvoie 0 en cas de succès, EOF en cas d'échec.